DAT42-1

Machine Learning II

Supervised models beyond the straight line: trees, forests, and boosting

---

One workflow, three new model families — and the judgment to choose between them.

Albert School — 22 hours

Contents

Preface

In **Machine Learning I** (DAT32-3) you learned to classify. Logistic regression gave you a probabilistic, interpretable model; you learned to read a confusion matrix, choose between accuracy, precision, recall, F1 and AUC-ROC according to the cost of false positives versus false negatives, tune a decision threshold, extend to multiple classes, scale and engineer features, and apply L1/L2 regularization. Before that, in **Intro to Machine Learning** (DAT32-1), you learned the project cycle itself: split the data, pick a baseline, train, evaluate on held-out data, diagnose, and retrain — all wired into a leak-proof scikit-learn pipeline and validated with cross-validation. This book assumes all of that. It does not re-teach the cycle; it swaps new models into it.

Logistic and linear regression share one hidden limitation: they draw a **straight** boundary. They assume the log-odds (or the target) move at a constant rate as each feature increases. Reality is rarely so polite. Rent levels off for large flats; churn risk spikes only when **both** tenure is low **and** recent usage has dropped. Capturing such bends and interactions with a linear model means hand-crafting polynomial and interaction features — guessing, in advance, the very shape you are trying to learn.

This course introduces three model families that learn that shape automatically. **Decision trees** carve the feature space into rectangular regions, bending and interacting features for free. They are simple, visual, and — on their own — unreliable, which is exactly why they are the perfect foundation. **Random forests** average many trees to cancel that unreliability. **Gradient boosting** grows trees in sequence, each correcting the last, and is the model that most often wins on tabular business data today.

Around these we gather the practical craft that makes them work: which algorithms need feature scaling and which are immune to it, how to engineer features the OP of DAT32-3 only hinted at (target encoding, date decomposition), how to search a hyperparameter space efficiently, and how to cope when one class is rare — the fraud, the churn, the defect. The course ends where every applied project ends: a single end-to-end pipeline, three competing architectures, and a written report that justifies the one you ship.

A word on method, unchanged from before: every idea arrives as a concrete example first and the general statement second. We keep one running problem throughout — **predicting which subscribers will churn next month** — a classification task you can already evaluate, so that every new model plugs into tools you already own.

1 From straight lines to rectangles: the decision tree

Logistic regression asks one question of an apartment-or-customer and weighs every feature at once. A decision tree asks a **sequence** of yes/no questions, like a flowchart, and each answer narrows down the prediction.

> **Worked example — a churn rule a manager might write by hand.** "If the customer's tenure is under 6 months, they are high-risk. Otherwise, if they used the product fewer than 3 times last month, they are also high-risk; if they used it 3 or more times, they are safe." That is a decision tree with two questions. Each question splits the customers into two groups; each final group gets a prediction. The tree **learns** which questions to ask, and in what order, from the data.

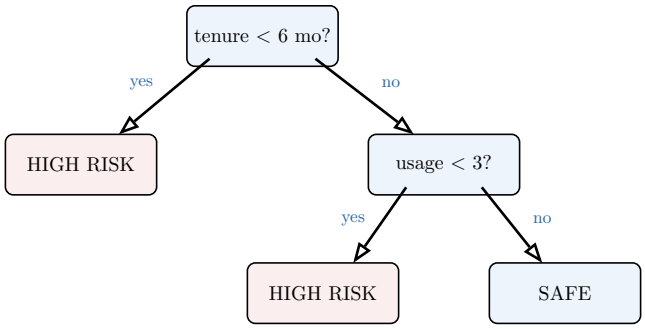

Figure 1: A decision tree as a flowchart. Each internal node tests one feature against one threshold; each branch is an answer; each leaf assigns a prediction. To classify a customer you start at the top and follow the answers down to a leaf. The tree's whole job is to discover good questions and thresholds from data.

Definition 1 A **decision tree** predicts by recursively splitting the data on one feature at a time. Each **internal node** tests a single feature against a threshold (e.g. `tenure < 6`), each **branch** is an outcome of that test, and each **leaf** holds a prediction. To predict for a new example, follow the tests from the **root** down to a leaf. A **classification tree** predicts the majority class of its leaf; a **regression tree** predicts the mean target of its leaf.

The geometry is the key to everything that follows. Because each test cuts on one feature, a tree partitions the feature space into **axis-aligned rectangles** — boxes whose sides are parallel to the axes. Logistic regression can only draw a single slanted line; a tree draws a staircase of boxes and assigns each box its own prediction.

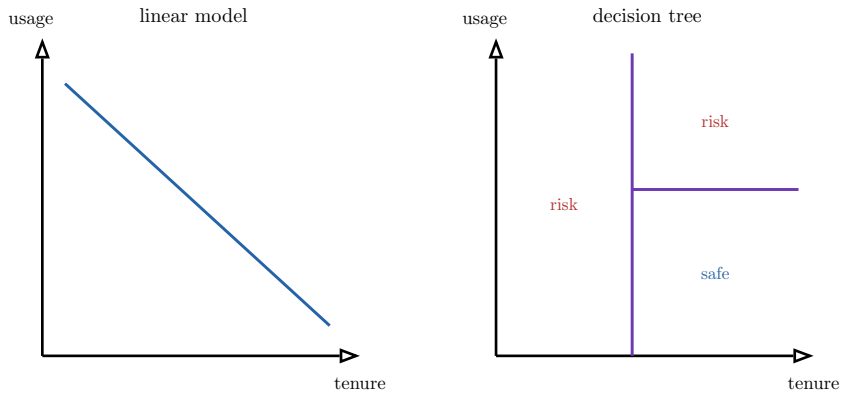

Figure 2: The same two-feature problem, two model shapes. The linear model (left) splits the plane with one slanted line. The tree (right) splits it into axis-aligned boxes with a sequence of one-feature cuts, giving each box its own label. The tree bends and interacts features for free — but its boundary is a staircase, never a clean diagonal.

This picture already explains two facts we will lean on for the rest of the course. First, a tree captures **interactions** automatically: the meaning of "low usage" depends on which tenure box you are in, because the second split happens **inside** the first. Second — and we return to this in the scaling chapter — a tree only ever compares one feature to a threshold, so **rescaling a feature changes nothing**: the split `tenure < 6 months` and `tenure < 180 days` separate the exact same customers.

1.1 How a split is chosen: impurity

The tree must decide, at each node, **which feature and which threshold** to split on. It does so greedily: it tries every candidate split and keeps the one that best **purifies** the resulting groups — that makes each child as close as possible to a single class (or, for regression, a single value).

> **Worked example — a good split versus a bad one.** A node holds 50 churners and 50 stayers — maximally mixed. Splitting on `tenure < 6` sends 40 churners + 5 stayers left and 10 churners + 45 stayers right: each side is now lopsided, almost pure. Splitting on `has_email = yes` sends 26 churners + 24 stayers left and 24 + 26 right: both sides are still a coin-flip. The first split is worth making; the second is worthless. We need a **number** that says so.

Definition 2 The **Gini impurity** of a node with class proportions $p_1, \ldots, p_K$ is

$$G = 1 - \sum_{k=1}^{K} p_k^2,$$

and the **entropy** is

$$H = -\sum_{k=1}^{K} p_k \log_2 p_k.$$

Both are zero when the node is pure (one class has $p_k = 1$) and maximal when the classes are balanced. They measure how mixed a node is.

Both functions peak when the node is a perfect mixture and fall to zero as it becomes pure. For two classes they trace almost the same arch, which is why the choice between them rarely changes the tree.

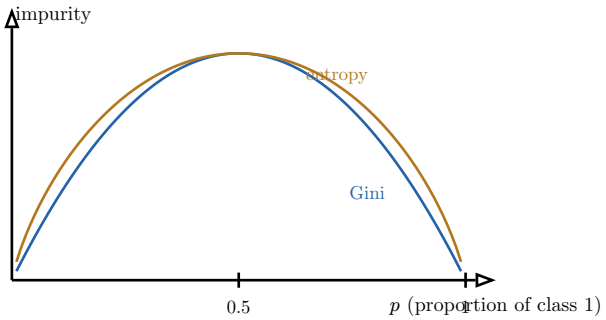

Figure 3: Gini impurity and entropy for a two-class node, against the proportion p of one class. Both are zero at the pure ends ($p = 0$ or 1) and peak at the even mixture $p = 0.5$. They have the same shape, so the splitting criterion almost never changes which tree you get; entropy is marginally costlier to compute because of the logarithm.

To score a split we compare the parent's impurity with the impurity of its children, weighted by how many examples fall into each.

Definition 3 The **information gain** of a split is the parent's impurity minus the size-weighted average impurity of the children:

$$\text{Gain} = I(\text{parent}) - \sum_{c \in \text{ children}} \left(\frac{n_c}{n}\right) I(c),$$

where I is Gini or entropy, n_c is the number of examples in child c and n the number in the parent. The tree chooses, at each node, the feature-and-threshold pair that **maximises** the information gain.

Worked example — scoring the good split. The parent (50/50) has Gini $1 - 0.5^2 - 0.5^2 = 0.5$. The good split's left child (40/5) has Gini $1 - (40/45)^2 - (5/45)^2 \approx 0.198$; the right child (10/45) has ≈ 0.298. Weighting by size (45/100 and 55/100) gives a child impurity of ≈ 0.254, so the gain is $0.5 - 0.254 = 0.246$. The worthless `has_email` split leaves both children near 0.5, for a gain of essentially zero. The tree keeps the split with the larger gain.

1.2 Regression trees: the same machine, a different ruler

For a continuous target the idea is identical, but "pure" no longer means "one class" — it means "all the targets in this node are close to their average." The natural impurity is then the variance, measured as mean squared error.

Definition 4 A **regression tree** splits to minimise the **mean squared error** within each child. A node's impurity is the MSE of its examples around their own mean,

$$\text{MSE}(\text{node}) = \frac{1}{n_{\text{node}}} \sum_{i \in \text{ node}} \left(y^{(i)} - \bar{y}_{\text{node}}\right)^2,$$

and a leaf predicts the mean target $\bar{y}_{\text{node}}$ of the examples that reach it. The split chosen is the one that most reduces the size-weighted MSE — exactly the information-gain rule with MSE in place of Gini.

So a single algorithm serves both tasks: classification swaps in Gini or entropy, regression swaps in MSE, and everything else — greedy search, recursive splitting, prediction at the leaves — is shared.

1.3 When to stop: depth and the overfitting trap

Left unchecked, a tree keeps splitting until every leaf is pure — one customer per leaf, if it must. That tree has **zero** training error and is worthless: it has memorised the data, the overfitting you met in DAT32-1 in its most extreme form.

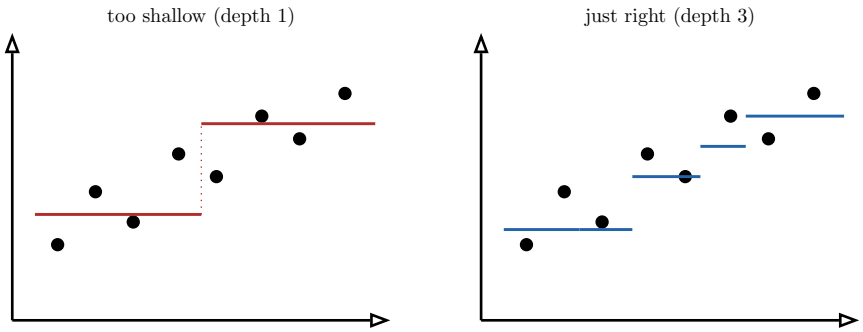

Figure 4: A regression tree is a staircase, and its depth sets the number of steps. Depth 1 (left) is one crude step — it underfits. Depth 3 (right) tracks the trend with a few well-placed steps. Keep adding depth and the staircase grows a step per point, threading the noise: textbook overfitting.

The cure is to **limit the tree's growth** with hyperparameters, then tune them by cross-validation exactly as you tuned λ in DAT32-3.

Definition 5 The growth of a tree is controlled by hyperparameters including the **maximum depth** (longest root-to-leaf path), the **minimum samples per leaf** (a leaf may not be smaller than this), and the **minimum samples to split** (a node below this size becomes a leaf). Smaller depth and larger minimum-size thresholds produce simpler trees that are less prone to overfit. These are **chosen** by cross-validation, not learned from the data.

Common pitfall A decision tree reported on its **training** accuracy looks magnificent — often 100%. That number is meaningless: an unconstrained tree can always memorise its training set. Judge every tree on held-out data, and assume any tree grown to full depth is overfit until cross-validation proves otherwise.

A single tree has a deeper problem that no amount of pruning fixes: it is **unstable**. Change a handful of training rows and a different feature may win the root split, sending the whole tree down a different path and producing wildly different predictions. That high variance is the weakness the next two chapters are built to cure — and, remarkably, the cure turns the weakness into a strength.

2 Random forests: many trees, averaged

A single deep tree has **low bias** (it can fit almost any shape) but **high variance** (it changes drastically with the data). DAT32-1 taught the cure for variance in one word: **average**. If we could train many trees and average them, the idiosyncratic errors — each tree's overreaction to its particular training rows — would partly cancel, while the real signal they agree on would survive.

Two obstacles stand in the way, and the random forest is precisely the pair of tricks that removes them.

2.1 Bagging: training on bootstrap samples

We have only one training set, so how do we get many **different** trees? We resample the data we have.

> **Definition 6** A **bootstrap sample** is drawn from a dataset of size n by sampling n rows **with replacement**: some rows appear several times, some not at all (about 37% are left out on average). **Bagging** (bootstrap aggregating) trains one model on each of many bootstrap samples and combines them — by **voting** for classification, **averaging** for regression.

Each bootstrap sample is a slightly different view of the data, so each tree overfits in its own direction. Averaging their predictions cancels those private errors. The mathematics is the same variance reduction that makes the mean of many measurements more reliable than any one: averaging B independent estimates, each with variance σ^2, gives variance σ^2/B.

> **Worked example — why averaging calms an unstable model.** Train 100 deep trees on 100 bootstrap samples of the churn data. On one borderline customer, 58 trees vote "churn" and 42 vote "stay." Any single tree might have landed either way — that is its high variance — but the **fraction** 0.58 is a stable estimate that barely moves if you redraw the bootstrap samples. The forest predicts "churn," and the 0.58 doubles as a calibrated-ish probability.

2.2 The forest's extra trick: random features

Bootstrap samples alone are not different enough. If one feature is very strong — say `tenure` — almost every tree will choose it for the root split, so the trees end up **correlated**, all making the same mistakes, and averaging correlated errors cancels little. The variance of an average of B estimates with pairwise correlation ρ is

$$\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2 :$$

the second term vanishes as B grows, but the first, $\rho\sigma^2$, does not. To drive variance down we must **decorrelate** the trees — shrink ρ.

> **Definition 7** A **random forest** is bagging of decision trees with one addition: at **each split**, only a random subset of the features (commonly $\sqrt{p}$ of the p features for classification) is considered as split candidates. This **feature subsampling** forces trees to use different features, decorrelating them so that averaging removes far more variance than bagging alone.

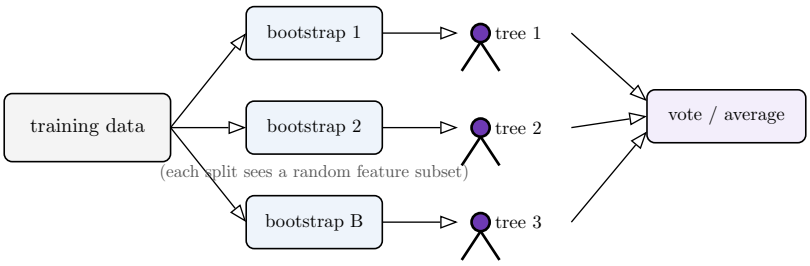

Figure 5: A random forest. Each tree is grown on its own bootstrap sample, and at every split it may only choose among a random subset of features. The two randomisations make the trees diverse; aggregating their votes (classification) or averages (regression) cancels their individual errors. More trees never hurt accuracy — they only cost compute.

Unlike a single tree, **adding more trees to a forest never causes overfitting** — it only stabilises the average. You stop adding trees when the score plateaus and the extra compute is no longer worth it, not to control complexity.

2.3 Reading a forest: feature importance

A forest sacrifices the single tree's one virtue — you can no longer trace a clean flowchart. But it offers a different kind of interpretability: it can rank **which features mattered**.

Definition 8 The **(impurity-based) feature importance** of a feature in a forest is the total reduction in impurity (Gini/MSE) contributed by all the splits on that feature, averaged over all trees and normalised to sum to one. A feature that produces large, frequent impurity drops scores high.

Common pitfall Impurity-based importance is **biased toward high-cardinality features** — those with many distinct values (IDs, raw timestamps, continuous variables) get more chances to split and so look spuriously important. When the ranking matters for a decision, prefer **permutation importance**: shuffle one feature's values in the validation set and measure how much the score drops. A feature whose shuffling barely hurts the model was not truly being used, whatever its impurity score claimed.

Importances tell you **what** the forest leaned on, not **why** or in which direction — they are a feature ranking, not the signed, "all else equal" coefficients of logistic regression. That trade — losing the readable coefficient, gaining accuracy and a robustness no single tree has — is the bargain of the whole tree-ensemble family.

3 Gradient boosting: trees that correct each other

A random forest builds its trees **in parallel and independently**, then averages them to cut variance. Gradient boosting takes the opposite stance: it builds trees **one at a time, in sequence**, each new tree trained to fix the errors the ensemble has made so far. Where bagging attacks **variance**, boosting attacks **bias** — it turns a crowd of weak, shallow trees into one strong predictor.

3.1 The core idea: fit the residuals

Start with a trivial prediction and improve it in small, targeted steps.

Worked example — boosting a regression by hand. Predict house prices. Step 0: predict the mean, €300k, for everyone. This is wrong by some **residual** for each house (+€80k here, −€50k there). Step 1: train a small tree not on the price, but on those **residuals** — it learns where the mean is too low or too high. Add a fraction of its predictions to the running total. The new residuals are smaller. Step 2: train another small tree on the **new** residuals. Repeat hundreds of times, each tree shaving off a little more error. The sum of all these small corrections is the boosted model.

Definition 9 **Gradient boosting** builds an additive model in stages. Writing F_m for the ensemble after m trees, each step fits a small tree h_m to the **negative gradient of the loss** (for squared-error regression, exactly the residuals $y - F_{m-1}(x)$) and updates

$$F_m(x) = F_{m-1}(x) + \nu\, h_m(x),$$

where $0 < \nu \leq 1$ is the **learning rate** (shrinkage). The "gradient" in the name is because fitting the negative gradient is gradient descent performed in the space of functions, one tree per step.

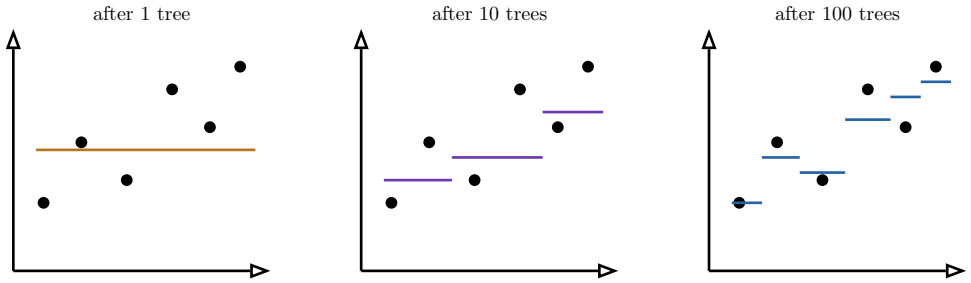

Figure 6: Boosting in stages. The first tree is a crude guess with large residuals; each subsequent tree fits what is left over, so the running sum bends closer to the data. Add too many trees, though, and the fit eventually chases noise — unlike a forest, a boosted model **can** overfit if you boost too long.

3.2 Bagging versus boosting: the contrast that organises everything

These two ensembles are mirror images, and holding the contrast in mind is the fastest way to reason about either.

	Bagging / random forest	**Boosting / GBM**
How trees are built	in parallel, independently	in sequence, each fixes the last
Each tree is	deep, low-bias, high-variance	shallow, high-bias, "weak"
Mainly reduces	variance	bias
More trees	never overfit — only plateau	**can** overfit — must be tuned
Key knobs	number of trees, features per split	learning rate, depth, number of trees
Temperament	robust, hard to misuse	accurate, needs careful tuning

Table 1: Bagging and boosting side by side. A forest averages independent deep trees to kill variance and is famously forgiving. A booster stacks dependent shallow trees to kill bias and usually wins on accuracy — at the price of genuine tuning. On most tabular business problems a tuned booster is the model to beat.

3.3 The libraries and their key hyperparameters

In practice you do not implement boosting from scratch; you use a tuned, regularised implementation. **XGBoost** and **LightGBM** are the two standard libraries — both build gradient-boosted trees, with engineering tricks for speed (LightGBM grows trees leaf-wise and bins continuous features; XGBoost adds explicit regularization to the split criterion). For learning, what matters is that they share the same handful of decisive hyperparameters.

Definition 10 The hyperparameters that govern a gradient-boosting model are, above all:

- the **learning rate** ν — how much of each tree is added; smaller is more accurate but needs more trees;
- the **number of trees** (`n_estimators`) — too few underfits, too many overfits;
- the **maximum depth** of each tree — boosters use **shallow** trees (often depth 3–8), since each tree need only be a weak learner;
- **subsampling** of rows and columns per tree — adds randomness that regularises;
- **L1/L2 regularization** on leaf values — the same penalties you met in DAT32-3.

The learning rate and the number of trees trade off against each other and are tuned together.

Common pitfall Lowering the learning rate while keeping the number of trees fixed makes a booster **worse**, not better — it underfits, because the trees no longer sum to a strong model. The two move together: halve the learning rate and you must roughly **double** the number of trees to compensate. Tune them as a pair, and use **early stopping** — halt when the validation score stops improving — to set the number of trees automatically.

4 Feature scaling: who needs it, who is immune

You met standardization and normalization in DAT32-3 and saw that they matter for logistic regression. Now that trees are in the toolkit, the rule sharpens into a clean dividing line — and it is one students routinely get wrong by scaling everything "to be safe."

Definition 11 **Feature scaling** puts features on a comparable numeric range. **StandardScaler** standardises each feature to zero mean and unit variance,

$$x' = \frac{x - \mu}{\sigma},$$

while **MinMaxScaler** rescales each feature to a fixed interval, usually $[0, 1]$,

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}.$$

Both are fitted on the **training data only** and then applied to validation and test data, inside the pipeline, to avoid leakage.

The decisive question is whether the algorithm compares features **by magnitude**.

> **Worked example — the same model, scaled and unscaled.** Train logistic regression on `income` (range 0–200,000) and `age` (range 18–90) without scaling: the gradient is dominated by income purely because its numbers are larger, and the L2 penalty punishes age's coefficient unfairly. Scale both to unit variance and the model improves at once. Now train a decision

tree on the identical unscaled data: it splits on `income < 50000` and `age < 30` regardless of scale — the result is **byte-for-byte identical** whether or not you scale. The tree never compares income to age; it only compares each feature to a threshold.

Need scaling (compare magnitudes)	**Immune to scaling** (split / threshold)
gradient descent models: linear & logistic regression	decision trees
regularized models (L1/L2 penalise raw coefficient size)	random forests
distance-based: k-NN, k-means	gradient boosting (XGBoost, LightGBM)
SVMs, neural networks, PCA	rule-based models generally

Table 2: The scaling dividing line. Anything that measures distances, sums weighted features, or penalises coefficient size is sensitive to the units of its inputs and must be scaled. Tree-based models compare one feature to a threshold at a time and are completely invariant to monotonic rescaling — for them, scaling is harmless but pointless.

Common pitfall "Scale everything just in case" is not free discipline — it wastes effort, adds a fragile step to your pipeline, and can mislead you into thinking scaling is **why** a tree model works when it changes nothing. Scale because the **algorithm** needs it (gradient-based, distance-based, regularized), not as a reflex.

5 Advanced feature engineering

DAT32-3 gave you polynomial features, interaction terms, and categorical encoding for linear models. Two of those carry over, and two new techniques — target encoding and date decomposition — round out the practical toolkit. Tree ensembles discover interactions on their own, so feature engineering shifts emphasis: less about helping the model bend, more about **handing it information it cannot extract from the raw columns** — especially high-cardinality categories and raw dates.

5.1 Polynomial and interaction features, revisited

Definition 12 **Polynomial features** add powers of a feature ($x^2, x^3, ...$) to let a model capture curvature. **Interaction terms** add products of features ($x_1 x_2$) to let the effect of one feature depend on another. For linear models these must be added by hand; tree ensembles capture both automatically through successive splits, so explicit polynomial and interaction features help linear models far more than they help trees.

5.2 Target encoding for high-cardinality categories

One-hot encoding, your default from DAT32-3, breaks down when a category has many values: a `zip_code` with 5,000 levels becomes 5,000 sparse columns. Target encoding replaces the category with a number that actually carries signal.

Definition 13 **Target encoding** (mean encoding) replaces a categorical value with the **mean of the target** for that category — e.g. each city's value becomes the churn rate

of customers in that city. It compresses a high-cardinality category into one informative numeric column.

Common pitfall Target encoding is a **leakage magnet**. If you compute a category's mean target using a row's **own** label, that row's answer leaks into its own feature and the model looks brilliant in training and collapses in production — most dangerously for rare categories seen only once, whose "encoding" is just their own label. Always target- encode **out-of-fold** (compute each fold's encoding from the **other** folds) and **smooth** rare categories toward the global mean. Treat target encoding as a model step inside the pipeline, never as a one-shot transform of the whole dataset.

5.3 Date decomposition

A raw timestamp is nearly useless to a model as a single big number, yet it hides some of the strongest features in business data.

Definition 14 **Date decomposition** expands a date or timestamp into its informative components — year, month, day-of-month, day-of-week, hour, and flags like **is_weekend** or **is_holiday**. Cyclical components (month, hour, weekday) are often encoded as $(\sin, \cos)$ pairs so that the model sees December and January, or 23:00 and 01:00, as **adjacent** rather than maximally far apart.

Worked example — unlocking a timestamp. A single column `2026-03-14 23:50` tells a churn model almost nothing as a raw number. Decomposed, it yields `weekday = Saturday`, `is_weekend = 1`, `hour = 23` — and now the model can learn that weekend-late-night sign-ups churn at a different rate. The cyclical $(\sin, \cos)$ encoding of the hour additionally tells it that 23:00 and 00:00 are an hour apart, not 23 hours apart.

6 Class imbalance

Many of the most valuable classification problems are **imbalanced**: fraud, churn, disease, and defects are rare. If 2% of customers churn, a model that predicts "no churn" for everyone scores 98% accuracy and is useless — it never catches the one case you care about. You already have the evaluation tools for this (precision, recall, F1, AUC-ROC from DAT32-3); this chapter adds the three techniques for **training** a model that actually finds the minority.

Worked example — the accuracy trap. A churn dataset is 2% churners. The "always predict majority" baseline gets 98% accuracy, 0% recall on churners — it catches none of them. Accuracy is the wrong metric here (you knew this from DAT32-3); recall, precision and AUC-ROC tell the real story. The fix is to make the rare class **count more** during training, by one of three routes.

6.1 Three techniques and their trade-offs

Definition 15 Three standard remedies for class imbalance:

1. **Class weighting** — multiply the loss for minority-class errors by a larger weight (e.g. `class_weight="balanced"`), so misclassifying a rare example hurts more. Cheap, built into most models, changes no data.
2. **SMOTE** (Synthetic Minority Over-sampling) — create **new** synthetic minority examples by interpolating between a minority point and its nearest minority neighbours, balancing the classes before training.
3. **Threshold adjustment** — train normally, then lower the decision threshold (from 0.5) so that more examples are predicted as the minority class, trading precision for recall.

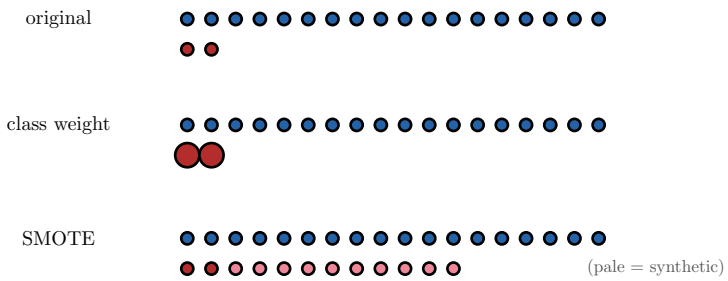

Figure 7: Three responses to a 18-to-2 imbalance. Class weighting (top) keeps the data but makes each minority error count more (drawn larger). SMOTE (bottom) manufactures synthetic minority points by interpolation until the classes balance. Threshold adjustment, not shown, leaves training untouched and instead moves the 0.5 cut-off after the fact.

Each technique buys recall at a different price, and choosing among them is the judgment the OP asks you to defend.

Common pitfall The trade-offs are not interchangeable. **Class weighting** is the safe default — it touches no data and rarely backfires. **SMOTE** can hurt: it invents data, may create implausible synthetic points in feature space, and — critically — **must be applied only to the training fold, never before the train/test split**, or synthetic points built from test neighbours leak the test set. **Threshold adjustment** changes no training at all and is often the cleanest fix when the model already ranks well (high AUC) but its default 0.5 cut-off is misplaced; recall its precision–recall cost from DAT32-3. A common, defensible recipe: train with class weights, then tune the threshold on validation to hit the precision/recall the business requires.

7 Hyperparameter tuning at scale

Every model in this course carries hyperparameters — a tree's depth, a forest's feature subset, a booster's learning rate and tree count. You tuned a single λ by hand in DAT32-3; with several interacting knobs, hand-tuning collapses, and you need a **search**. Two strategies dominate, and knowing when each is preferable is an explicit learning outcome.

Definition 16 **Grid search** defines a finite set of values for each hyperparameter and evaluates **every combination** by cross-validation — exhaustive and reproducible, but its cost **multiplies** with each added hyperparameter (the "curse of dimensionality": 4 knobs $\times$ 5 values each $=$ 625 fits). **Randomized search** instead **samples** a fixed number of combinations at random from the (possibly continuous) ranges — you control the budget

directly, and it explores more values of each individual hyperparameter for the same number of fits.

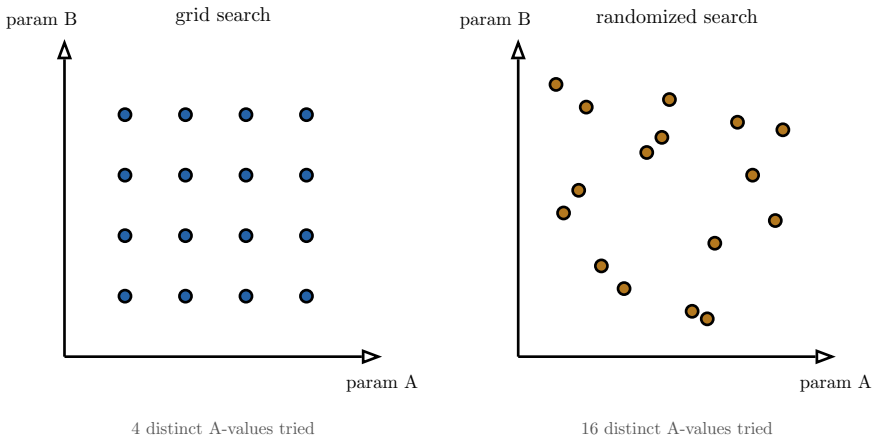

Figure 8: Grid versus randomized search, same 16-fit budget. The grid (left) tries only 4 distinct values of each parameter. If `param A` matters and `param B` barely does, the grid has wasted three-quarters of its fits re-testing useless B-values at each A. Random search (right) tries 16 distinct values of the **important** parameter, so it explores the dimension that matters far better.

So: **prefer grid search** when you have **few** hyperparameters, each with a **small** known set of sensible values, and you want exhaustive, reproducible coverage. **Prefer randomized search** when the space is **large** or **continuous**, when you have a **fixed compute budget**, or when you suspect only a few hyperparameters truly matter — which, for tree ensembles, is the usual case. A common workflow uses a broad randomized search to locate a good region, then a small grid to refine within it.

8 Putting it together: the end-to-end pipeline and model comparison

The course converges on a single deliverable that exercises everything above: take raw data and produce a **validated model with a written evaluation report comparing three architectures**. The discipline is the leak-proof pipeline of DAT32-1, now carrying every tool from this course.

Definition 17 A **complete supervised ML pipeline** chains, as one fitted object: preprocessing (imputation, scaling **where the model needs it**, encoding including out-of-fold target encoding, date decomposition), any imbalance handling (applied to training folds only), the model, and hyperparameter search by cross-validation — all sealed so the **test set is touched exactly once**, at the very end, for the reported number.

The comparison study is where judgment shows. You fit three architectures on the **same** split with the **same** preprocessing and the **same** cross-validation, so the contest is fair.

Worked example — a fair three-way contest. For the churn problem, compare **logistic regression** (the interpretable DAT32-3 baseline — scaled, regularized), a **random forest** (robust, little tuning), and a **gradient-boosting model** (tuned via randomized search). Score all three by the metric the business chose — say AUC-ROC and recall at a fixed precision — on identical cross-validation folds. Suppose the booster scores AUC 0.91, the forest 0.88,

logistic regression 0.84. The booster wins on accuracy, but if the business needs to **explain** each churn flag to a retention team, logistic regression's readable coefficients may still be the right ship-it choice. The report must state the metric, the scores, **and** the non-accuracy factors (interpretability, training cost, inference speed, maintenance) that justify the final pick.

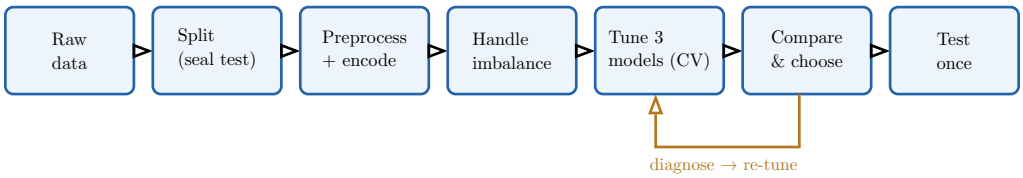

Figure 9: The end-to-end pipeline as a deliverable. It is the DAT32-1 workflow with this course's tools slotted in: encoding and date decomposition in preprocessing, imbalance handling on training folds only, three tuned architectures compared on identical folds, and the test set unsealed only at the final step. The written report documents the comparison and defends the final choice on more than accuracy alone.

The arc of this course. A decision tree carves the feature space into rectangles, choosing splits that most reduce impurity (Gini/entropy for classification, MSE for regression) — flexible, visual, but unstable and prone to overfit. A **random forest** averages many decorrelated trees to crush that variance and ranks features by importance. **Gradient boosting** stacks shallow trees in sequence, each correcting the last, to crush bias — usually the most accurate model on tabular data, at the cost of real tuning. Around these: scale features only for magnitude-sensitive models (not trees); engineer features the model cannot extract itself (target encoding, date decomposition); handle rare classes with weighting, SMOTE, or threshold adjustment, each with its own trade-off; and search hyperparameters with grid (small spaces) or randomized (large spaces) search. The whole toolkit converges on one leak-proof pipeline and a reasoned, documented choice among three competing architectures.

Exercises

1. Implement a decision tree **classifier** and a decision tree **regressor** on a dataset of your choice. Visualise each tree, and for one internal node compute the Gini impurity (or MSE) of the parent and children by hand to confirm the library's chosen split maximises information gain.

2. Train a single deep tree and report both its training and validation accuracy. Explain the gap. Then constrain `max_depth` and `min_samples_leaf` by cross- validation and show how the validation score and the train/validation gap change.

3. Contrast Gini and entropy as splitting criteria on the same data: fit a tree with each and report whether the resulting trees and their validation scores differ. Relate your finding to the shape of the two impurity curves.

4. Implement a random forest. Demonstrate variance reduction by comparing the prediction stability of a single tree versus the forest across several bootstrap resamples of the training data. Explain the role of feature subsampling in decorrelating the trees.

5. Extract feature importances from your forest two ways: impurity-based and permutation-based. Construct a case (e.g. include a high-cardinality ID column) where the two rankings disagree, and explain which to trust and why.

6. Train a gradient-boosting model (XGBoost or LightGBM). Explain in your own words how boosting differs from bagging. Then sweep the learning rate and number of trees **together**, showing that lowering the learning rate requires more trees to maintain performance.
7. Take one tree-based model and one gradient-based model (logistic regression). Train each with and without `StandardScaler`. Show that scaling is irrelevant for the tree and consequential for logistic regression, and explain the reason from how each algorithm uses its features.
8. Engineer features from raw variables: add polynomial and interaction terms to a linear model and measure their effect; target-encode a high-cardinality category **out-of-fold**; and decompose a timestamp into calendar components. Demonstrate the target-encoding leakage failure by encoding in-fold and showing the inflated training score.
9. Take an imbalanced dataset and handle it three ways — class weighting, SMOTE, and threshold adjustment. Report precision, recall, F1 and AUC-ROC for each, and write a paragraph on the trade-off of each technique. Show explicitly why SMOTE must be applied only inside the training folds.
10. Tune the same model with both grid search and randomized search under an equal compute budget. Compare the best score each finds and the number of distinct values of the most important hyperparameter each explored. State when you would prefer each strategy.
11. Build a complete end-to-end pipeline from raw data to a validated model, with the test set sealed until the final evaluation. Compare three architectures (logistic regression, random forest, gradient boosting) on identical cross-validation folds, and write an evaluation report that justifies your final choice on accuracy **and** on interpretability, training cost, and inference speed.